A New Heuristic for Minimizing Schedule Length in Heterogeneous Computing Systems

D.Sirisha G.Vijaya Kumari
Department of Computer Science and Engineering
JNTUH College of Engineering, JNTU
Hyderabad, India
sirishad998@gmail.com

***Abstract*— Heterogeneous environments includes resources with diverse capabilities necessitates efficient task-to-processor assignment for accomplishing high performance. In the proposed work a new heuristic approach for scheduling the tasks independent of their levels in the precedence constrained task graph is detailed. Schedules with shorter span are achieved by reducing the start time of the independent tasks. The proposed algorithm is compared with the available literature and an improvement in the schedule length is obtained.**

Keywords— task scheduling; precedence constraint task graph; heuristics; schedule length; heterogeneous computing systems.

I. INTRODUCTION

Computationally complex problems can be competently solved by dividing them a set of dependent tasks that can be executed in parallel. Heterogeneous Computing Systems (HCS) have evolved as an economical alternative to cater the various computing needs. HCS are primarily composed of resources with diverse capabilities inter-connected by high speed networks. Efficient scheduling of tasks is imperative for achieving promising potentials in HCS.

Scheduling strategies sort the tasks and map each task to the most appropriate processor. The problem of task scheduling is generally shown to be NP-Complete [1] and can be solved in polynomial time if P=NP. Generally, heuristics are used to generate near optimal solutions.

Scheduling algorithms for HCS are categorized into static and dynamic. Static scheduling requires the entire information for scheduling such as execution time of tasks, communication times among the tasks and the topology of the application must be known before hand [3, 9-12]. Static time heuristics are further classified as list based scheduling algorithms [2, 4-9], task duplication based algorithms [3,8,10], clustering algorithms and guided random search methods. List scheduling heuristics prioritizes the tasks and on this basis declares the order of execution of tasks and allocates these tasks to the processor which finishes them earlier. List scheduling heuristics generally produce good schedules with less complexity.

In the present work, a new heuristic namely Minimizing Schedule Length (MSL) for the list based task scheduling in HCS is proposed. The article is further classified as follows. Section II defines the task scheduling problem with the required terminology. The available literature is presented in section III. MSL algorithm is detailed in Section IV. In Section V, the performance of MSL algorithm is evaluated with the related algorithms used for comparison. Finally, the summary of the research work is presented in section VI.

II. THE TASK SCHEDULING PROBLEM

Task scheduling model for HCS includes three major components namely, an application represented as precedence constraint task (PCT) graph, a HCS and a strategy for scheduling.

A. Application Model

An application with a set of tasks having dependencies among themselves is taken as input for scheduling problem. The problem is designed on a graph G. G = (V, E) where V is a task set and E is a set of edges. Each edge connecting two tasks t_i and t_j, where $i \neq j$, imposes precedence constraint between the tasks t_i and t_j such that the task t_j starts only after t_i is completed. The weight of the directed edge between the tasks t_i and t_j signifies the communication time $c_{i,j}$ required for transferring the data between the two tasks. And $c_{i,j}$ is 0, when the two tasks are performed on same processor. A task becomes independent once its precedence constraints are satisfied. An independent task becomes ready for execution when it receives the data from all its predecessor tasks. The predecessor tasks of t_i are represented as $pred(t_i)$ and the successors of t_i are represented as $succ(t_i)$.

A task which has no predecessors is termed as *begin task* and which has no successors is *end task*. Generally, a PCT graph has one begin task and one end task. Contrarily, more than one begin and an end task necessitates the inclusion of pseudo task with zero execution time connected with zero communication time edges.

B. Resource Model

A HCS include a set of m processors with diverse computing speeds for executing an application with n tasks designed as PCT graph. The execution time of n tasks on m processors is represented by n x m matrix, where each element $e_{i,j}$ signifies the estimated time for executing the task t_i on processor p_j. The communication time between the tasks is represented by n x n matrix, where each element $c_{i,j}$ is the time to transmit the data from t_i to t_j tasks. All the processors are fully connected. Moreover, the tasks are executed without any preemption. An example PCT graph and execution time matrix are presented in Fig.1 and Table 1.

978-1-4799-6085-9/15/$31.00©2015 IEEE

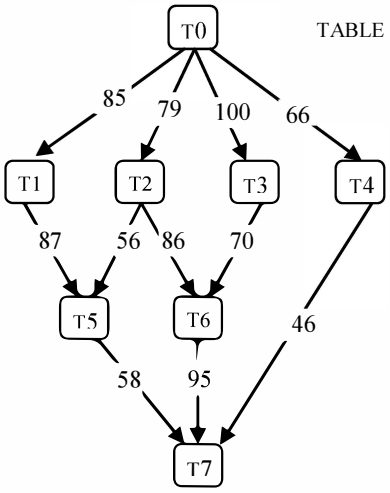

Fig. 1. Application PCT graph

TABLE 1.EXECUTION TIME MATRIX

Task	P1	P2
T0	70	84
T1	68	49
T2	78	96
T3	89	26
T4	30	88
T5	66	86
T6	25	21
T7	96	26

According to [6], the task scheduling problem is defined using $EBT(t_i,p_j)$ and $ECT(t_i, p_j)$. $EBT(t_i,p_j)$ is the earliest begin time of task t_i on processor p_j. It is computed by considering the processor free time and task ready time, is computed using (2), for an begin task t_{begin},

$$EBT(t_{begin}, p_j) = 0 \quad (1)$$

$$EBT(t_i, p_j) = max\{ free[p_j], \max_{tm \in pred(ti)} (AFT(t_m)+c_{m,i}) \} \quad (2)$$

where $t_m \epsilon pred(t_i)$ is the immediate predecessors of t_i, $avail[p_j]$ is the time the processor p_j becomes free to execute t_i. The inner max block gives the ready time of t_i i.e., time its immediate predecessors are completed and the data from them is available to the task. $AFT(t_m)$ is the actual finish time of t_i's predecessors i.e., t_m. $c_{m,i}$ is the time for communicating the data from each predecessor task t_m to t_i. And $c_{m,i}$ is zero if t_i and its predecessor task t_m are allocated to the same processor p_j.

The ECT of a task t_i on processor p_j represented as $ECT(t_i, p_j)$, is the time t_i completes its execution on processor p_j, it is computed using (3).

$$ECT(t_i, p_j) = EBT(t_i, p_j) + e_{i,j} \quad (3)$$

where $EBT(t_i,p_j)$ is computed as shown in equation (2) and $e_{i,j}$ is the executing time of t_i on p_j. After scheduling all tasks in PCT graph, the schedule length also termed as makespan is the AFT of the end task t_{end}, defined as

$$makespan = max\{AFT(t_{end})\} \quad (4)$$

The objective of the problem is to minimize the makespan of an application.

III. RELATED WORK

Scheduling the tasks in a PCT graph on HCS has been widely studied. Several heuristics are developed to reduce the makespan. The efficiency of list scheduling algorithms can be exploited by the heuristics, employed for prioritizing each task. The list based scheduling algorithms available in the literature include Heterogeneous Earliest Finish Time (HEFT) [2], Critical Path On a Processor (CPOP) [2], Performance Effective Task Scheduling (PETS) [4], Longest Dynamic Critical Path (LDCP) [5], Dynamic Level Scheduling (DLS) [6], Levelized Minimum Time (LMT) [7] and Mapping Heuristic (MH) [8]. Topcuoglu et al. [2] proposed HEFT and CPOP algorithms and in their research work evaluated HEFT, CPOP, DLS, MH and LMT strategies. They concluded that HEFT is better performing heuristic than the rest. Ilavarasan et al. [4] presented a low complexity PETS algorithm and demonstrated through experiments that PETS generated better schedules than HEFT, CPOP and LMT algorithms.

The better performing heuristics in the literature are HEFT, CPOP and PETS algorithms. Thus, these heuristics become the base for the proposed work to achieve much better scheduling result. The HEFT, CPOP and PETS algorithms are briefly discussed below.

A. HEFT Algorithm

HEFT [2] scheduling strategy is of two phases. The first phase prioritizes the tasks on the basis of upward rank. The upward rank for each task t_i is the sum of the average execution and communicating times down the lengthiest path (critical path) from t_i to end task. The second phase maps each task on the processor with gives least ECT.

B. CPOP Algorithm

CPOP algorithm also performs in two phases. The first phase assigns priority to the tasks using two attributes namely upward rank and downward rank. CPOP algorithm adapts similar strategy used by HEFT algorithm to compute upward rank. The downward rank is the critical path length from the begin task to the task, excluding the execution time of the task. In the second phase, the algorithm adapts similar approach in assigning the tasks to the processor as HEFT algorithm except for the critical path tasks which are allocated to the processor that minimizes the execution time of all critical path tasks.

C. PETS Algorithm

The scheduling strategy of PETS [4] algorithm performs in three phases. In the first phase, tasks at each level are grouped to perform them in parallel. The second phase assigns priority to each task based on its rank which is computed using the parameters such as mean execution time of a task, communication time to transmit the data from a task to its immediate successors and the maximum rank among the its predecessors. In the third phase, the PETS algorithm adapts the strategy of HEFT for executing the task.

IV. THE PROPOSED SCHEDUING STRATEGY

Much of the list based scheduling strategies available in the literature order the tasks level – wise [2,4], the order of the execution of the tasks in general is considered level wise i.e., to

perform the tasks in parallel. Generally, lower level tasks acquire higher priority than tasks at higher level. Tasks at the same level are sorted according to their priority. However, the tasks are not necessarily uniform and hence a priority / weightage may be assigned to the individual tasks. The priority scheduling is expected to enhance the performance. The priority to be assigned to each individual task is not deviated. The heuristic is believed to enhance the performance of the scheduling. The improvement in the performance can be achieved through the identification of parallel tasks which are not necessarily in the same level. In much of the earlier scheduling schemes, parallelism was considered only amongst the tasks in the same level.

For the PCT graph in Fig.1, the execution of the tasks begins with begin task *t0*. With this, the tasks *t1, t2, t3* and *t4* dependent on *t0* become independent as their precedence constraints are satisfied. The execution of tasks *t1, t2* prior to tasks *t3, t4* satisfies the precedence constraints for the task t5 at level 2. Level-wise algorithms imposed level-constraints on *t5*, hence it has to wait until tasks *t3, t4* at level 1are executed. On scrutinizing these tasks, there is no dependency between the tasks *t3, t4* at level 1 and the task *t5* at level2. Likewise, with the completion of task *t3* at level 1, task *t6* at level 2 becomes independent. The tasks *t5* and *t6* at level 2 may be scheduled without imposing any constraints on the completion of the tasks in the previous level. On the other hand, the proposed heuristic considers parallelism of all tasks whose precedence constraints are satisfied independent of the level at which they exist.

The EBT of the tasks t_5, t_6 and t_7 when PETS HEFT and CPOP algorithms are employed is always greater. The EBT of these tasks for PETS and HEFT algorithms is 230, 300 and 420 time units while it is 237, 206 and 379 time units for CPOP algorithm. However, by employing the MSL algorithm the EBT of these tasks is reduce to 204, 237 and 357 time units. The reduction in the EBT of the independent tasks leads MSL strategy to generate better schedules. The proposed scheduling approach can be applied for computationally intensive problems that can be broken down into a set of dependent tasks. Executing such tasks on HCS can speed up the completion of an application.

The proposed MSL algorithm works in two phases namely the task ordering phase and task mapping phase. The task ordering phase defines the order of processing the tasks. The task mapping phase determines the best processor for executing the task. The two phases are detailed below.

A. Task Ordering Phase

The task ordering phase assigns priority to the tasks and then arranges them in the order of their priority preserving their precedence constraints. Priorities to the tasks can be assigned based upon their rank values. The rank of a task can be computed using two attributes namely mean execution time (MET) and total communication time (TCT). The MET for each task is defined as the mean execution time of a task on *m* processors, and is computed using (6).

$$MET(t_i) = \sum_{j=1}^{m} e_{ij} / m \quad (5)$$

where $1 \le i \le n$, n is the number of tasks and $1 \le j \le m$, m is the number of processors.

The TCT of a task is defined as the sum of the time required for receiving the data from its immediate predecessors and the time required for transmitting the output data from a task to its immediate successors. The TCT for an intermediate task t_j is computed using (7).

$$TCT(t_j) = \sum_{i=1}^{p} c_{i,j} + \sum_{k=1}^{q} c_{j,k} \quad (6)$$

where $1 \le i \le p$, p is the number of predecessors of t_j and $1 \le k \le q$, q is the number of successors of t_j.

$$TCT(t_j) = 0 + \sum_{k=1}^{q} c_{j\,k}; \quad \text{for the begin tasks} \quad (7)$$

$$TCT(t_j) = \sum_{i=1}^{p} c_{i,J} + 0; \quad \text{for the end tasks} \quad (8)$$

The rank of each task is computed using MET and TCT values of a task and is defined as follows.

$$rank(t_i) = MET(t_i) + TCT(t_i) \quad (9)$$

where $1 \le i \le n$, n is the number of tasks.

The ranks for the tasks in PCT graph are computed. A ready task list is constructed and initialized with the begin task. The tasks which become independent are inserted into ready list. An independent task is the one which has all its predecessor tasks processed. The tasks in the ready list are positioned in the decreasing order of their ranks. The task with highest rank receives utmost priority. If more than one task has equal rank then the task with maximum MET value is selected. The attribute values for computing the priority to the tasks for the PCT graph given in Fig.1 are presented in Table 2.

TABLE 2. THE PRIORITY COMPUTATION TABLE

Task	MET	TCT	RANK	Priority
0	77	330	407	1
1	58	172	230	3
2	87	224	311	2
3	57	170	227	5
4	59	112	171	7
5	76	200	276	4
6	23	251	274	6
7	61	199	260	8

TABLE 3. The schedule generated by MSL algorithm in each iteration

Step	Ready Tasks	Highest Priority Task	p_1		p_2		Best Processor
			EBT	ECT	EBT	ECT	
1	t_0	t_0	0	70	0	84	p_1
2	t_2, t_1, t_3, t_4	t_2	70	148	149	245	p_1
3	t_1, t_3, t_4	t_1	148	216	155	204	p_2
4	t_5, t_3, t_4	t_5	291	357	204	290	p_2
5	t_3, t_4	t_3	148	237	290	316	p_1
6	t_6, t_4	t_6	237	262	307	328	p_1
7	t_4	t_4	262	292	290	378	p_1
8	t_7	t_7	348	444	357	383	p_2

The MSL scheduling strategy for the given PCT graph in the Fig.1 begins with the single task in the ready list i.e., the begin task t_0. Therefore, t_0 attains priority 1. Table 3 presents the tasks in the ready list and the higher priority task is selected in each iteration. Upon the completion of the task t_0, tasks t_1, t_2, t_3 and t_4 in the level 1 become independent and are subsequently inserted into the ready list. The priorities to the tasks in the ready list are assigned on the basis of rank values. Among all the tasks in the ready list task t_2 possessing maximum rank hence receives priority 2. From the remaining tasks in the ready list, t_1 emerges as the task with highest rank and attains priority 3. With the completion of the tasks t_1 and t_2, the precedence constraints for the task t_5 are satisfied. Although task t_5 belongs to next level, it gets privileged by MSL scheduling strategy and is inserted into the ready list. The task t_5 now competes with the t_3 and t_4 tasks in the ready list and receives maximum priority i.e., 4. It can be observed from table 2 that task t_3 has higher rank than t_4 and hence receives priority 5. The task t_6 becomes independent as its predecessor tasks t_2 and t_3 are executed. Task t_6 now competes with t_4 in the ready list and achieves priority 6 while t_4 attains priority 7. Finally, the end task t_7 attains the position 8. The task sequence obtained by MSL scheduling strategy for the PCT graph given in Fig.1 is 0-2-1-5-3-6-4-7. The makespan of the given PCT graph is the *ECT* (t_7), and it is 383 time units.

B. Task Mapping Phase

The input to the task mapping phase is the task sequence obtained by the task ordering phase. The ECT for each task in the task sequence is computed on all the processors. The processor with minimum ECT is selected for mapping the task. The selected processor is marked as the best processor for executing the task. This phase employs insertion based policy [12] which checks for holes in the schedule and examines the possibility of inserting the task in the hole if sufficient hole is available preserving the precedence constraints. If such a hole is not available, the task is scheduled after the last task on a processor. If the ECT of a task is same for two processors, then lightly loaded processor is selected. Table 3 presents the best processor selected for each task given PCT graph of Fig.1.

The proposed MSL algorithm is presented below.

Algorithm 1 The MSL algorithm

Algorithm MSL (T, E, P, e, c)
Input: A PCT graph G = (V, E) with $n \in$ V tasks and $e \in$ E edges.
// $p \in$ P is a set of processors.
// c[1: n,1: n] is the communication time adjacency
// matrix of a n task graph such that $c_{i,j}$ is a positive
// e[1: n,1: p] is the execution time matrix such that
// $e_{i,j}$ is the execution time of the task t_i on processor p_j.
1. Compute MET and TCT values.
2. Compute rank = MET + TCT, for each task in the PCT graph.
3. Construct a ready list L and initialize with begin task.
4. enqueue (t_i) // insert the independent tasks into the ready list L.
5. Sort the tasks in the non-increasing order of rank values in the ready list.
6. while L not empty do
7. delete (t_i, L) //delete the highest priority from L.
8. for each processor $p_j \in$ P do
9. Calculate ECT(t_i, p_j) using insertion based policy
10. end for
11. Allocate the task t_i to the processor p_j with minimum ECT(t_i,p_j).
12. Update ready list with the successors of t_i.
End while.

Fig.2 The proposed MSL Scheduling Algorithm

The time complexity of MSL algorithm is as follows. The ranking of the tasks requires a time complexity of $O(n + e)$. Each task is inserted into the ready list implemented using binary heap with a complexity of $O(\log n)$. $O(n \log n)$ steps are required for ordering the tasks according to their priority. The overall time complexity for task ordering phase is $O(n \log n + e)$. The task mapping phase requires a complexity of $O((n + e)\, p$. Thus, the time complexity of MSL strategy is $O((n \cdot \log n)p+e)$.

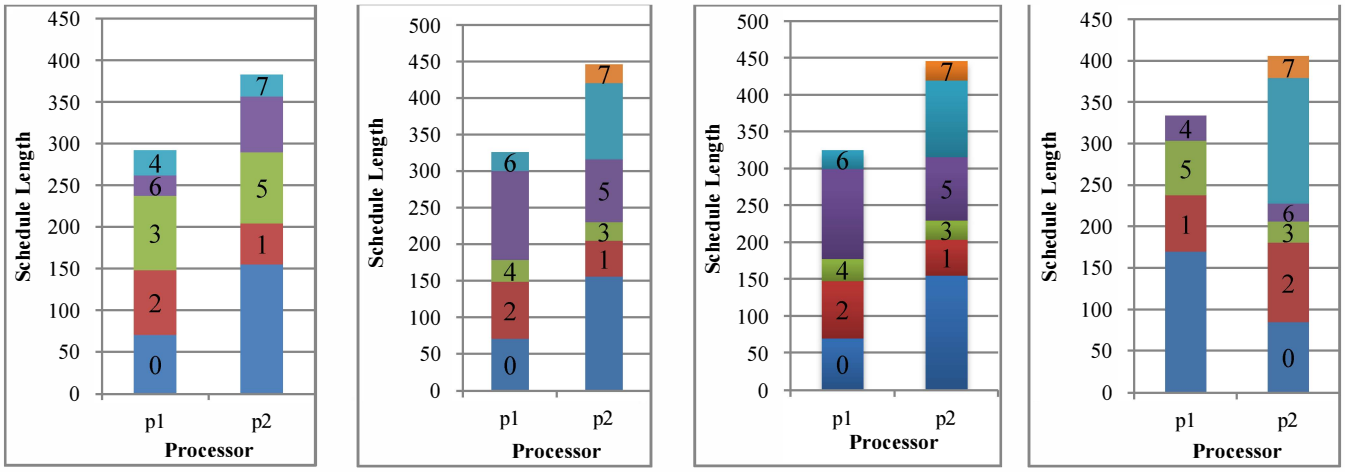

Fig. 3. Schedules generated by (a) MSL (makespan =383), (b) PETS (makespan = 446), (c) HEFT (makespan = 446) and (d) CPOP (makespan = 405) algorithms for the PCT graph given in Fig.1.

V. EXPERIMENTION RESULTS AND ANALYSIS

The proposed MSL algorithm is compared with the algorithms discussed in the related work. The performance of the scheduling algorithms is evaluated by the experimentation on randomly generated PCT graphs using the following metrics as cited in [2, 8].

1) Makespan Ratio (MR): The makespan of an algorithm for a PCT graph is compared to its lower bound *lb*, and is defined as MR. The *lb* of the makespan of PCT graph is computed by the sum of the lowest execution time of each task on critical path. The scheduling algorithm with lowest MR for a graph tends to produce better makespan. The MR of a PCT graph is defined using (10)

$$\text{MR} = \frac{\text{makespan returned by an algorithm}}{lb} \tag{10}$$

2) Speedup : This metric defines the gain in the execution time accomplished though parallelization of tasks on HCS consisting of processors, m = {4, 8, 12, 20} compared to the sequential execution time obtained by executing all the tasks in PCT graph on one processor. The scheduling algorithm with higher speedup is considered as better, it is computed using (11)

$$\text{Speedup} = \frac{\text{sequential execution time}}{\text{parallel execution time}} \tag{11}$$

3) Efficiency: Defined as the speedup achieved forvaried processor set m = {4,8,12,20} used in the HCS for scheduling the PCT graph, is computed using (12)

$$\text{Efficiency} = \frac{\text{Speedup}}{\text{number of processors used}} \tag{12}$$

4) Execution Time of Algorithm: The time taken by each algorithm to generate the output schedule for a PCT graph.

A. Random PCT Graphs

A set of random PCT graphs are generated using various input parameters [2] described as follows.

- A set of tasks n in a PCT graph. The n value is varied from 40 to 100 with an increment of 20.
- Communication to execution ratio (CER): CER is the ratio of mean communication time to mean execution time of a PCT graph. A very low CER value signifies a computationally intensive application. The CER values set for the experimentation are 0.1, 0.5, 1.0, 5.0 and 10.0.
- Parallelization factor (α): The PCT graph is generated with l number of levels and each level contains t number of tasks. The value of l can be determined by $\sqrt{n} / \alpha$, for n tasks in a graph. The t value for each level is computed by $\sqrt{n} \times \alpha$. For the experimentation the value of α is varied from 0.5 to 1.5 with an increment of 0.5. When α value is greater than 1.0 a shorter graph with maximum parallelism and when α value is less than 1.0 longer graph with minimum parallelism can be generated.
- Heterogeneity factor (γ): The γ factor indicates the variation in the task execution times on p processors. When γ value is high the deviation in the task execution times among the processors is significant. And low γ value implies slight deviation in the task execution times.
- The execution time e_{ij} of each task n_i on processor p_j is random integer chosen from the range:

$$MET \times (1-\gamma/2) \leq e_{ij} \leq MET \times (1+\gamma/2) \tag{13}$$

where γ is the heterogeneity factor of the graph and MET is the mean execution time of a task. The MET values considered for experiments are {30, 40, 50, 60, 80, 100}. The experimentation using simulation are conducted on 4320 PCT graphs obtained from the combination of above mentioned parameters to prevent bias towards a particular scheduling algorithm.

B. Performance Analysis

The first experiment set compares average MR and speedup values with various graph sizes. Each data point plotted in the Fig. 4(a) and 4(b) is the averaged data obtained from 1080 experiments. It can be observed from Fig. 4(a) that the average MR values for all the algorithms is linearly growing with the increase in the number of tasks and MSL algorithm has lesser average MR value than HEFT, PETS and CPOP algorithms. The experimental results reveal that MSL algorithm generated shorter schedules than HEFT by 7.09%, PETS algorithm by 8.87% and CPOP algorithm by 12.01%. Fig.4(b) shows the speedup for various graph sizes. The proposed algorithm has also demonstrated considerable improvement in the speedup by 9.44%, 10.97% and 15.83% compared to PETS, HEFT and CPOP algorithms.

Fig. 4(c) and 4(d) depict makespan of the scheduling algorithms as a function of CER and graph structure value. Each data point reported in the Fig.4(c) is the averaged data from 864 experiments. From the Fig.4(c), it is evident that all the algorithms are prone to CER values. The rise in the average MR values of the algorithms is more evident with the increase in the CER values. For CER $\leq$ 1, MSL algorithm generates the lowest average MR value compared with the HEFT, PETS and CPOP algorithms by 5.16%, 6% and 8.69% respectively. Moreover, the proposed algorithm demonstrated better average MR values for CER > 1 by 8.43%, 10.68% and 14.36% relative to HEFT, PETS and HEFT strategies. The performance of all algorithms declines as CER value increases. The superior performance of the MSL algorithm for various CER values can be attributed to the algorithms' competence in handling heavily dependent tasks. The MSL algorithm identifies heavily communicating and computing tasks and assigns higher priority to these tasks and schedules them ahead of the other tasks.

The next experiment compares the average MR value with parallelization factor α, is shown in Fig. 4(d). Each data point plotted in the Fig. 4(d) is the averaged data from 1800 experimentations. When α value is 0.5, MSL algorithm showed better average MR compared to HEFT, PETS and CPOP algorithms by 8.27%, 10.41% and 13.7% respectively. When α value is equal to 1.5, MSL algorithm showed superior performance than HEFT, PETS and CPOP algorithms by 5.88%, 7.42% and 10.19% respectively. On average, MSL algorithm surpassed HEFT, PETS and CPOP algorithms by 6.89%, 8.27% and 11.46%. The performance improvement of the proposed MSL algorithm is achieved for all α values as it releases the tasks soon after its predecessors are executed and places them in the ready list. Hence, the number of tasks in the ready list of MSL algorithm is always more than HEFT, PETS and CPOP algorithms. Thus MSL algorithm increases the parallelization of the tasks on HCS and consequently reduces the makespan of graph.

Further, the algorithms competence is evaluated by scheduling the PCT graphs on a set of processors p = {4, 8, 12, 20}. Fig. 4(e) shows the decline in the average efficiency for all the algorithms as the number of processors is increased. MSL strategy shows better efficiency than HEFT, PETS algorithm and CPOP algorithms by 9.51%, 11.31% and 15.43% respectively. With respect to average execution time, it can be observed from Fig.4 (f) that MSL is the fastest and CPOP is the slowest algorithm. MSL algorithm finished earlier than HEFT by 20.56%, PETS by 32.89% and CPOP algorithm by 63.07%.

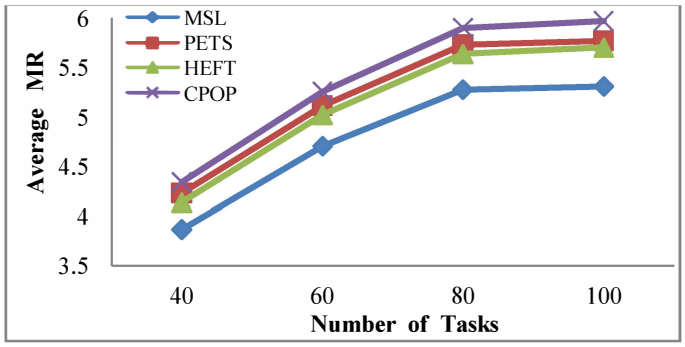

Fig. 4(a)

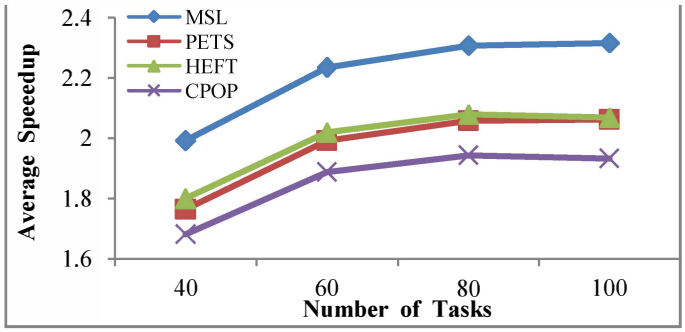

Fig. 4(b)

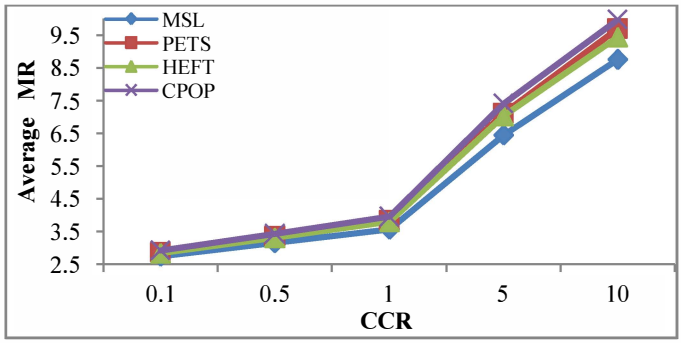

Fig. 4(c)

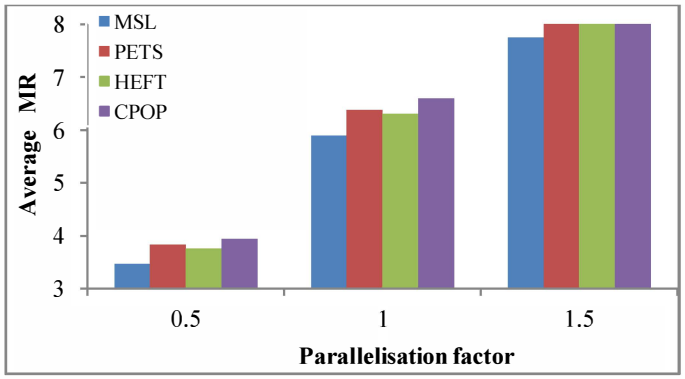

Fig. 4(d)

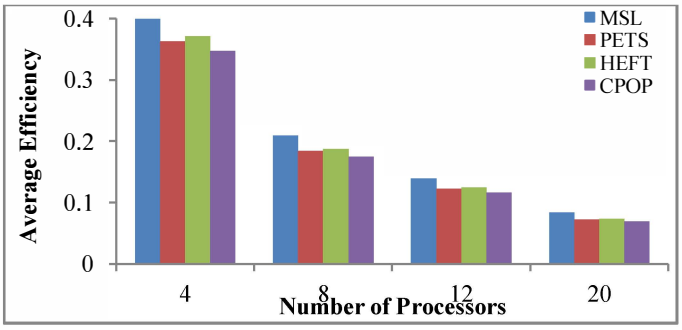

Fig. 4(e)

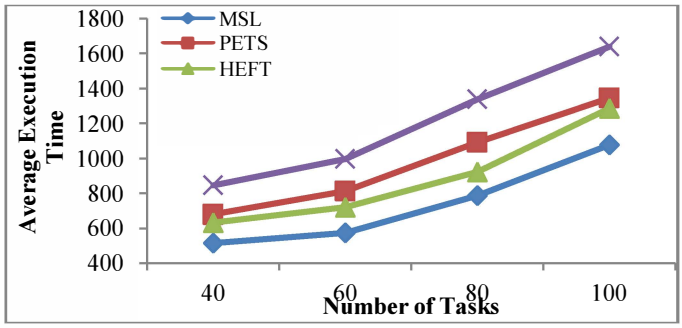

Fig. 4(f)

VI CONCLUSION

A new heuristic namely MSL scheduling strategy for HCS is proposed in this paper. The scheduling scheme devised groups the tasks whose precedence constraints are satisfied independent of their level. The minimization of the EBT of the independent tasks is achieved by eliminating the level-wise constraints; this potentially leads the MSL scheduling strategy in generating shorter span schedules. Experiments are conducted to analyze the performance of the proposed scheduling strategy with the existing PETS, HEFT and CPOP algorithms. And the results of the experimentation reveal that MSL algorithm generates shorter length schedules than PETS, HEFT and CPOP algorithms. Moreover, MSL algorithm showed considerable performance improvement for higher CCR and parallelization factor values.

REFERENCES

[1] M.R. Gary, D.S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W.H. Freeman and Co., San Francisco, CA, 1979.

[2] H.Topcuoglu, S. Hariri, M.Y. Wu, "Performance effective and low complexity task scheduling for heterogeneous computing", IEEE Trans. Parallel Distributed Systems, 13 (3), 260–274, 2002.

[3] Sanjeev Baskiyar and Christopher Dickinson, "Scheduling directed a-cyclic task graphs on a bounded set of heterogeneous processors using Task Duplication", Journal of Parallel and Distributed System, vol. 65, pp. 911-921, May 2005.

[4] E. Illavarasan and P.Thambidurai, "Low complexity performance effective task scheduling algorithm for heterogeneous computing environments", J. Computer Science, 3(2): 94-103, 2007.

[5] Mohammad I. Daoud, and Nawwaf Kharma, "A high performance algorithm for static task scheduling in heterogeneous distributed computing systems", Journal of Parallel Distributed Computing, 68: 399-409, 2008.

[6] G.C. Sih, E.A. Lee, "A compile time scheduling heuristic for interconnection constrained heterogeneous processor architectures", IEEE Trans. Parallel Distributed Systems, 4 (2), 175–187,1993.

[7] H El-Rewlni and T.G. Lewis. "Scheduling parallel programs onto arbitrary target machine", Journal of Parallel and Distributed Computing, vol. 9.no. 2, pp. 138-153, June 1990.

[8] S. Bansal, P. Kumar, K. Singh, "Dealing with heterogeneity through limited duplication for scheduling precedence constrained task graphs", Journal of Parallel Distributed Computing. 65 (4),479–491, 2005.

[9] Georffey Falzon, Maozhen Li. "Enhancing list scheduling heuristics for dependent job scheduling in grid computing environments", Journal of Super Computing, vol.59, issue 1, pp. 104-130, Jan 2012.

[10] R. Bajaj, D.P. Agrawal, "Improving scheduling of tasks in a heterogeneous environment", IEEE Trans. Parallel Distributed Systems, 15 (2), pp. 107–118, 2004.

[11] M A Khan, "Scheduling for heterogeneous systems using constrained critical paths", Journal of Parallel Computing, vol.38, pp.175-193, 2012.

[12] Arabnejad, H., Barbosa, J. M., "List Scheduling Algorithm for Heterogeneous Systems by an Optimistic Cost Table", IEEE Trans. Parallel Distributed Systems,vol.25, no.3, pp.682-694, March 2014.